

Lab. 2 – Criação de Classificadores baseado em Dados

CMC-13 – Introdução a Ciência de Dados

Lucas Saorin

João Victor Saraiva Rodrigues

Fabio Eiiti Kobayashi Urio

25 de junho de 2026

Sumário

1	Preparação dos Dados	3
1.1	Descrição do Dataset	3
1.2	Limpeza Inicial	3
1.3	Codificação de Variáveis Categóricas	3
1.4	Análise Exploratória	4
1.5	Investigação de Problemas	4
1.5.1	Separação Perfeita em <code>Contract Length</code>	4
1.5.2	Granularidade de <code>Total Spend</code>	5
1.6	Análise de Correlação	5
1.7	Normalização e Exportação	6
2	Criação dos Modelos de Classificação	6
2.1	Metodologia Comum	6
2.2	SVM (Support Vector Machine)	6
2.2.1	Configuração	7
2.2.2	Justificativa dos hiperparâmetros	7
2.3	MLP (MultiLayer Perceptron)	7
2.3.1	Arquitetura e Configuração	7
2.3.2	Justificativa dos hiperparâmetros	7
2.4	AdaBoost	8
2.4.1	Configuração	8
2.4.2	Justificativa dos hiperparâmetros	8
3	Análise Comparativa do Desempenho dos Modelos	8
3.1	Métricas utilizadas	8
3.2	Por que Balanced Accuracy ao invés de F1-Score	9
3.3	Resultados	9
3.4	Discussão	10

4	Conclusões, Comentários e Sugestões sobre o Trabalho	10
4.1	Conclusões	10
4.2	Comentários	11
4.3	Sugestões de Melhoria	11

1 Preparação dos Dados

1.1 Descrição do Dataset

O conjunto de dados tem como objetivo prever a rotatividade de clientes (*customer churn*). A variável alvo é **Churn** (0 = cancelou o serviço, 1 = não cancelou). Os dados foram fornecidos em dois arquivos separados:

- **Treino:** 440.832 amostras $\times$ 12 colunas
- **Teste:** 64.374 amostras $\times$ 12 colunas

As features originais são: **CustomerID**, **Age**, **Gender**, **Tenure**, **Usage Frequency**, **Support Calls**, **Payment Delay**, **Subscription Type**, **Contract Length**, **Total Spend** e **Last Interaction**.

A taxa de churn no treino é de 56,7% e no teste de 47,4%, indicando uma diferença de distribuição entre os conjuntos que, como será discutido adiante, influenciou diretamente o desempenho dos modelos.

1.2 Limpeza Inicial

Foram aplicadas as seguintes transformações antes da análise exploratória:

1. **Remoção de linhas vazias:** encontrada 1 linha completamente vazia no conjunto de treino (índice 199.295), removida via `dropna(how='all')`.
2. **Remoção de CustomerID:** identificador único sem valor preditivo.
3. **Conversão float $\rightarrow$ int no treino:** artefato da linha NaN removida. Após a remoção, colunas que deveriam ser inteiras passaram a ter dtype `float64`. A conversão explícita restaurou o tipo correto e alinhou a granularidade com o conjunto de teste.

1.3 Codificação de Variáveis Categóricas

Três variáveis categóricas foram codificadas antes da exportação dos dados:

- **Gender:** codificação binária (Male = 0, Female = 1).
- **Contract Length:** codificação ordinal respeitando a ordem natural de duração (Monthly = 1, Quarterly = 3, Annual = 12).
- **Subscription Type:** *one-hot encoding* com `drop_first=True`, gerando as colunas `Subscription Type_Premium` e `Subscription Type_Standard`. A categoria *Basic* torna-se a referência implícita.

1.4 Análise Exploratória

A análise exploratória abrangeu histogramas, curvas KDE e boxplots para as sete features contínuas (Age, Tenure, Usage Frequency, Support Calls, Payment Delay, Total Spend, Last Interaction), sempre com treino e teste lado a lado para detectar eventuais diferenças de distribuição.

A Figura 1 exibe a distribuição de cada feature segmentada por classe de churn. Observa-se que **Support Calls** e **Total Spend** apresentam a separação mais nítida entre as classes: clientes que cancelaram tendem a ter mais chamadas de suporte e menor gasto total. **Payment Delay** e **Age** mostram separação moderada; as demais features apresentam distribuições muito sobrepostas.



Figura 1: Distribuição das features por classe de churn (Churn=0 vs Churn=1) no conjunto de treino.

1.5 Investigação de Problemas

1.5.1 Separação Perfeita em Contract Length

Durante a análise de distribuição por churn, identificou-se um comportamento crítico em **Contract Length**: contratos *Monthly* apresentam taxa de churn de **100%** no conjunto de treino (87.104 amostras), porém apenas **51,6%** no conjunto de teste (22.130 amostras). Quarterly e Annual mantêm comportamento consistente entre treino e teste ($\approx 46\%$).

Essa discrepância caracteriza um *distribution shift* severo: qualquer modelo aprende a regra “Monthly = churn garantido” e erra metade das predições no teste para contratos mensais. A Figura 2 ilustra essa diferença.

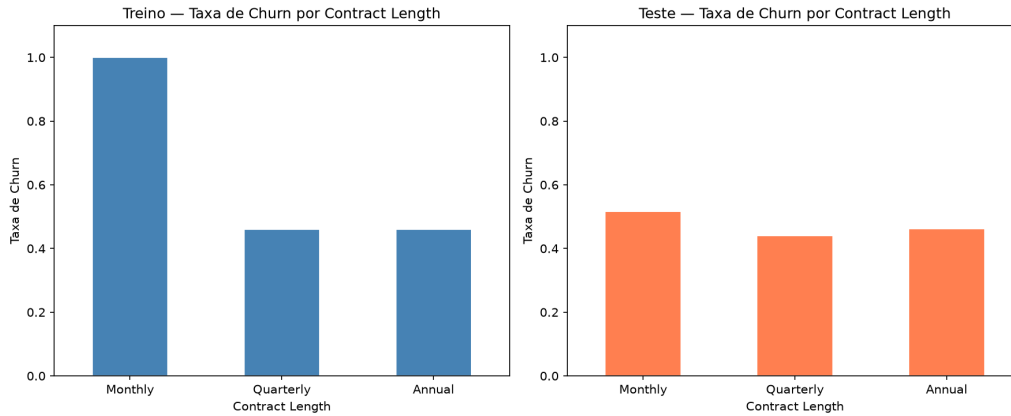


Figura 2: Taxa de churn por tipo de contrato no treino (esquerda) e no teste (direita). O comportamento de *Monthly* diverge completamente entre os conjuntos.

Um teste adicional confirmou o impacto: uma árvore de decisão com profundidade 3 utilizando apenas **Support Calls** e **Contract Length** atingiu 84,3% de acurácia no treino, mas somente 61,7% no teste (queda de 23 pp). Com todas as features, o treino sobe para 92,2% e o teste cai para 58,5% — adicionar mais features piorou a generalização, evidenciando memorização do treino.

Decisão: **Contract Length** foi removida antes da exportação dos dados. Essa foi a melhoria de maior impacto no projeto.

1.5.2 Granularidade de Total Spend

O conjunto de treino original apresentava **Total Spend** em formato `float64` (valores como 932.0, 185.3), enquanto o teste já estava em inteiros. A conversão `float` → `int` unificou a granularidade: ambos os conjuntos passaram a ter 901 valores únicos inteiros no intervalo [100, 1000].

1.6 Análise de Correlação

A correlação Point-Biserial entre cada feature e a variável alvo (Figura 3) identificou três grupos:

- **Features fortes** ($|r| > 0,3$): **Support Calls** ($r = 0,574$), **Total Spend** ($r = -0,429$), **Payment Delay** ($r = 0,312$).
- **Features moderadas** ($0,1 < |r| < 0,3$): **Age** ($r = 0,218$), **Gender** ($r = 0,175$), **Last Interaction** ($r = 0,150$).
- **Features fracas** ($|r| < 0,1$): **Tenure**, **Usage Frequency** e ambas as dummies de **Subscription Type** (correlação $\approx 0,01$). A taxa de churn é quase idêntica nas três categorias de assinatura (≈ 56 – 58%), indicando baixo poder discriminativo.

A comparação entre Pearson, Spearman e Kendall mostrou coeficientes consistentes para todas as features, confirmando que a relação entre as features e o churn é predominantemente linear.

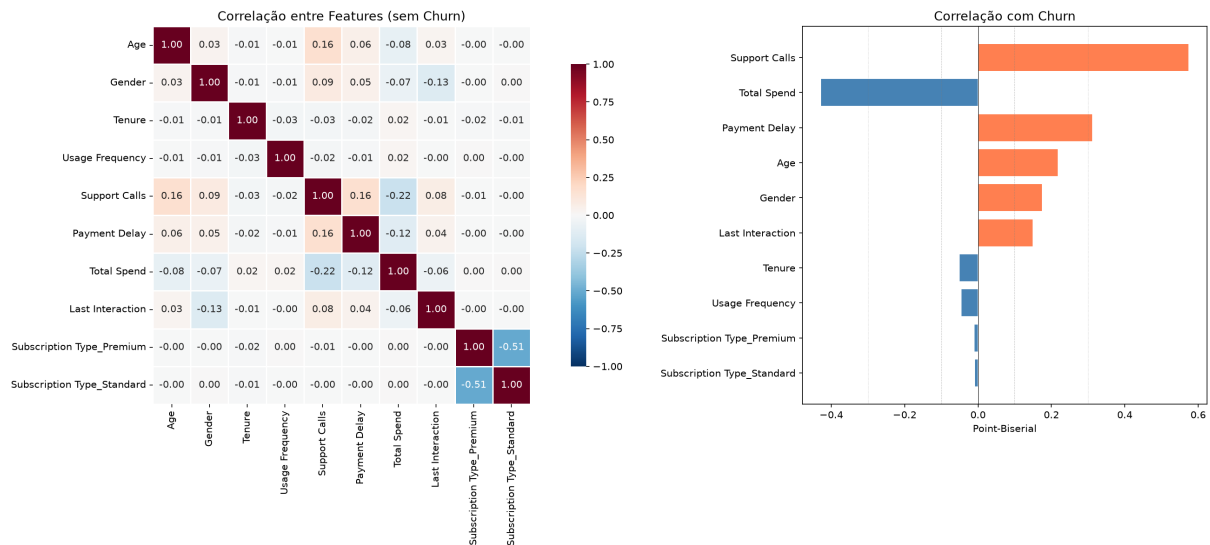


Figura 3: À esquerda: matriz de correlação entre features. À direita: correlação Point-Biserial de cada feature com a variável alvo **Churn**.

1.7 Normalização e Exportação

Após todas as transformações, o conjunto final conta com **10 features**. Um **StandardScaler** foi ajustado exclusivamente no conjunto de treino e aplicado ao teste (sem vazamento de informação). Os dados escalados foram exportados no formato *Parquet* e reutilizados por todos os notebooks de modelagem.

2 Criação dos Modelos de Classificação

2.1 Metodologia Comum

Todos os modelos seguiram o mesmo protocolo de treinamento e avaliação:

1. O conjunto de treino (440.832 amostras) foi dividido em X_{tr} (80%) e X_{val} (20%), com estratificação pela variável alvo.
2. Cada modelo foi treinado em X_{tr} .
3. O **threshold de decisão** foi calibrado no X_{val} por meio da curva Precision-Recall: encontra-se o limiar que maximiza o F1 no conjunto de validação. Isso evita usar o threshold fixo 0,5, que se mostrou inadequado dado o desbalanceamento residual entre treino e teste.
4. A avaliação final foi realizada no conjunto de teste (64.374 amostras), nunca utilizado durante o desenvolvimento.

2.2 SVM (Support Vector Machine)

O SVM é um classificador baseado em margens: encontra o hiperplano que separa as classes com a maior margem possível no espaço de features.

2.2.1 Configuração

Como o `LinearSVC` do `scikit-learn` não fornece probabilidades de saída, ele foi encapsulado em um `CalibratedClassifierCV` com validação cruzada de 3 folds, o que adiciona a capacidade de estimar `predict_proba` e viabiliza a calibração do `threshold`.

Tabela 1: Hiperparâmetros do SVM.

Parâmetro	Valor
Tipo	LinearSVC + CalibratedClassifierCV
C (regularização)	0,1 (regularização forte)
class_weight	balanced
max_iter	2000
cv (calibração)	3 folds

2.2.2 Justificativa dos hiperparâmetros

O valor $C = 0,1$ impõe regularização forte, penalizando menos o erro de classificação em favor de uma margem mais ampla, o que tende a melhorar a generalização diante de datasets com shift de distribuição. O `class_weight='balanced'` ajusta os pesos das classes automaticamente para compensar o desbalanceamento entre as taxas de churn no treino (56,7%) e no teste (47,4%).

2.3 MLP (MultiLayer Perceptron)

A rede neural *Multilayer Perceptron* é um classificador que aprende representações hierárquicas das features por meio de camadas de neurônios com funções de ativação não lineares.

2.3.1 Arquitetura e Configuração

Tabela 2: Configuração da rede MLP.

Parâmetro	Valor
Camadas ocultas	2 (64 $\rightarrow$ 32 neurônios)
Função de ativação	ReLU
Solver	Adam
Learning rate inicial	0,001
Regularização L2 (α)	0,01
Early stopping	Sim (10% do X_tr como validação interna)
Máx. de épocas	100

2.3.2 Justificativa dos hiperparâmetros

A arquitetura compacta (64 $\rightarrow$ 32) foi escolhida para equilibrar capacidade de aprendizado e risco de overfitting dado o tamanho do dataset. A regularização L2 com $\alpha = 0,01$ penaliza pesos grandes, reduzindo o overfitting. O *early stopping* interrompe o treinamento

quando o desempenho no conjunto de validação interna para de melhorar, evitando épocas desnecessárias.

A Figura 4 exibe a curva de perda ao longo das épocas de treinamento.

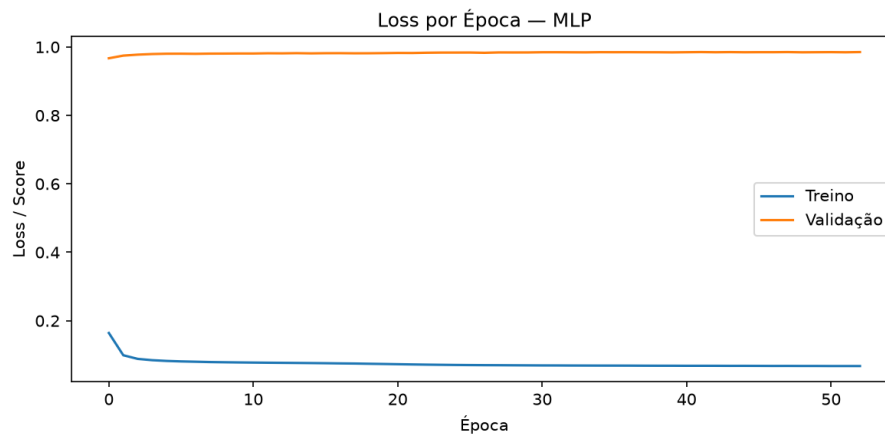


Figura 4: Curva de loss por época do MLP. O early stopping interrompe o treinamento quando o ganho adicional torna-se irrelevante.

2.4 AdaBoost

O AdaBoost (*Adaptive Boosting*) é um método de comitê que combina múltiplos classificadores fracos de forma sequencial: cada estimador foca nos exemplos que o anterior classificou incorretamente, aumentando seus pesos. O resultado final é uma combinação ponderada de todos os estimadores.

2.4.1 Configuração

Tabela 3: Hiperparâmetros do AdaBoost.

Parâmetro	Valor
Número de estimadores	200
Learning rate	0,05
Estimador base	DecisionTreeClassifier (stump, max_depth=1)

2.4.2 Justificativa dos hiperparâmetros

O `learning_rate` = 0,05 é conservador: cada estimador contribui menos para a combinação final, forçando um aprendizado mais gradual e estável. Combinado com 200 estimadores, esse ajuste favorece a convergência suave em detrimento de uma convergência rápida que poderia levar a overfitting.

3 Análise Comparativa do Desempenho dos Modelos

3.1 Métricas utilizadas

Foram calculadas seis métricas para cada modelo no conjunto de teste:

- **Balanced Accuracy:** $\frac{\text{Recall}_0 + \text{Recall}_1}{2}$. Métrica de ranking principal neste trabalho (ver justificativa abaixo).
- **Acurácia:** proporção de predições corretas.
- **Precision:** proporção de positivos preditos que são verdadeiros positivos.
- **Recall:** proporção de positivos reais corretamente identificados.
- **F1-Score:** média harmônica de Precision e Recall.
- **Kappa de Cohen:** concordância entre predições e gabarito descontando o acaso.

Dois classificadores triviais (*Dummy*) foram incluídos como linha de base: *most_frequent* (prevê sempre a classe majoritária) e *stratified* (prevê aleatoriamente respeitando a distribuição do treino). Um modelo real só é útil se superar esses baselines.

3.2 Por que Balanced Accuracy ao invés de F1-Score

Com recall próximo de 1,0 e precision próxima de 0,5, o F1-Score fica artificialmente inflado mesmo para modelos sem poder discriminativo real. O Dummy que prevê sempre churn=1 obtém $F1 = 0,643$, valor quase idêntico ao MLP ($F1 = 0,663$) — o F1 não distingue o modelo do acaso nesse cenário. A Balanced Accuracy não é afetada por esse viés: o mesmo Dummy obtém 0,500 (aleatoriedade pura), enquanto um modelo real deve superar esse valor.

3.3 Resultados

A Tabela 4 apresenta o desempenho de todos os classificadores avaliados, ordenados por Balanced Accuracy. A Figura 5 exibe o comparativo visual.

Tabela 4: Desempenho dos modelos no conjunto de teste (64.374 amostras), ordenados por Balanced Accuracy.

Modelo	Bal. Acc.	Acurácia	Precision	Recall	F1	Kappa
SVM	0,604	0,584	0,533	0,984	0,691	0,200
AdaBoost	0,576	0,554	0,515	0,997	0,679	0,146
Árvore de Decisão	0,576	0,554	0,515	0,996	0,679	0,145
KNN	0,568	0,545	0,510	0,992	0,674	0,129
Random Forest	0,543	0,519	0,496	0,998	0,663	0,082
XGBoost	0,543	0,519	0,496	0,998	0,663	0,081
MLP	0,542	0,518	0,496	0,999	0,663	0,080
Dummy (most_freq)	0,500	0,474	0,474	1,000	0,643	0,000
Dummy (stratified)	0,499	0,495	0,472	0,566	0,515	-0,003

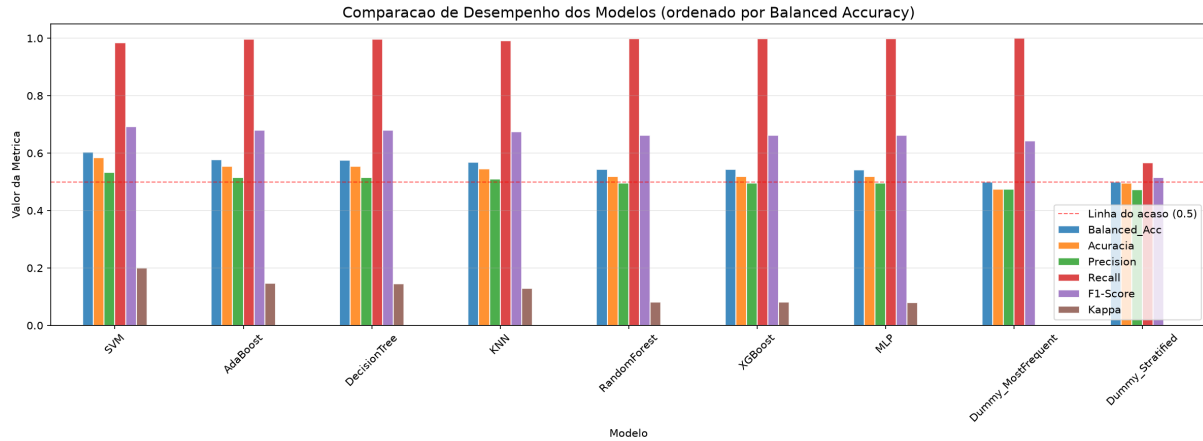


Figura 5: Comparação das métricas de todos os modelos, ordenados por Balanced Accuracy. A linha vermelha tracejada marca o nível do acaso (0,500).

3.4 Discussão

Todos os modelos reais superam os baselines. A Balanced Accuracy de todos os sete classificadores treinados está acima de 0,500 (limiar dos Dummies), confirmando que os modelos aprenderam algum padrão genuíno nos dados.

O SVM apresentou o melhor desempenho geral, com Balanced Accuracy = 0,604 e Kappa = 0,200. Esse resultado está alinhado com o perfil dos dados: as correlações Point-Biserial mostraram que a relação entre as features e o churn é predominantemente linear, contexto em que classificadores baseados em hiperplano tendem a se sair bem.

AdaBoost ocupou a segunda posição (Balanced Acc = 0,576, Kappa = 0,146), igualando a Árvore de Decisão simples. A sequência de estimadores fracos não trouxe ganho expressivo sobre o modelo base neste dataset.

MLP foi o pior dos três modelos escolhidos (Balanced Acc = 0,542, Kappa = 0,080). A rede neural exige mais dados ou features com padrões não-lineares relevantes para superar modelos lineares; neste problema, o sinal disponível parece insuficiente para explorar a capacidade extra da MLP.

O recall elevado ($> 0,98$) em todos os modelos indica que os thresholds calibrados ainda são baixos (em torno de 0,35–0,45). Isso decorre de um distribution shift residual: mesmo após a remoção do **Contract Length**, as distribuições de **Support Calls** e **Payment Delay** ainda diferem entre treino e teste (notado na EDA), fazendo com que as probabilidades estimadas pelos modelos no conjunto de teste sejam sistematicamente mais altas do que o real.

4 Conclusões, Comentários e Sugestões sobre o Trabalho

4.1 Conclusões

O SVM com regularização forte ($C = 0,1$) e threshold calibrado mostrou-se o modelo mais robusto para o problema de predição de churn neste dataset, atingindo Balanced Accuracy = 0,604 e Kappa = 0,200 no conjunto de teste.

O principal desafio do trabalho não foi técnico, mas estrutural: o dataset apresenta um *distribution shift* severo na variável **Contract Length**, onde contratos *Monthly* têm 100% de churn no treino mas comportamento radicalmente diferente no teste. Identificar e tratar esse problema foi mais impactante do que qualquer escolha de hiperparâmetro.

Os modelos mais simples (SVM linear, Árvore de Decisão) superaram os mais complexos (MLP, Random Forest, XGBoost), o que é consistente com o perfil do problema: as relações entre features e churn são predominantemente lineares, e os dados de teste têm uma distribuição ligeiramente diferente do treino, favorecendo modelos com menor capacidade de memorização.

4.2 Comentários

A métrica F1-Score, utilizada inicialmente como critério de ranking, se mostrou inadequada neste contexto: com recall $\approx 1,0$ e precision $\approx 0,5$, o F1 de um Dummy que prevê sempre a classe positiva atingiu 0,643 — valor próximo ao dos modelos reais. A substituição pelo Balanced Accuracy como métrica principal revelou com mais clareza o nível real de discriminação de cada classificador.

A calibração de threshold via curva Precision-Recall no conjunto de validação foi uma melhoria metodológica relevante: eliminou a dependência do limiar fixo de 0,5 e permitiu que cada modelo encontrasse seu ponto de equilíbrio próprio entre Precision e Recall.

4.3 Sugestões de Melhoria

- **Adversarial Validation:** treinar um classificador binário “é amostra de treino ou de teste?”. Se o AUC for alto, há shift detectável que merece investigação antes da modelagem.
- **Busca de hiperparâmetros:** substituir valores fixos por `RandomizedSearchCV` com validação cruzada interna, garantindo que as escolhas sejam orientadas por evidência e não por julgamento manual.
- **Feature engineering:** criar features de interação como `Support Calls × Payment Delay` pode capturar padrões que as features isoladas não expressam.
- **Investigação da origem do shift:** a discrepância em `Contract Length` pode indicar um problema na coleta dos dados (viés de amostragem ou mudança temporal no perfil dos clientes) que deveria ser corrigido na fonte antes da modelagem.
- **Calibração de probabilidades (Platt Scaling):** além do threshold, calibrar as probabilidades brutas dos modelos melhoraria a qualidade das previsões probabilísticas e permitiria comparações mais justas entre classificadores.